Topic: Unit 1
BCA 6TH SEM DS2 ML

Prepared By:
Dr Tejinder Kaur
Associate Professor

Data Visualization:

Data visualization uses charts, graphs and maps to present information clearly and simply. It turns complex data into visuals that are easy to understand. With large amounts of data in every industry, visualization helps spot patterns and trends quickly, leading to faster and smarter decisions.

What is Data Visualization?

- Turning data into visual formats like charts, graphs, and maps
- Makes complex data easy to understand at a glance
- Helps identify trends, patterns, and insights

- **Common Types of Data Visualization**
- There are various types of visualizations where each has a unique purpose in data representation. Here are the most common types:
- **Charts and Graphs:** They are used to visualize data, with charts comparing data points across categories or showing trends over time and graphs analyzing relationships between variables to identify correlations, trends and outliers. Examples: Bar Charts, Line Charts, Pie Charts, Scatter Plots, Histograms, Box Plots.
- **Maps**: They are used to display geographical data which provides spatial context to trends and patterns. Examples: Geographic Maps, Heat Maps
- **Dashboards:** They combine multiple visualizations into a single interface which provides real-time insights and interactive features for users to explore data.

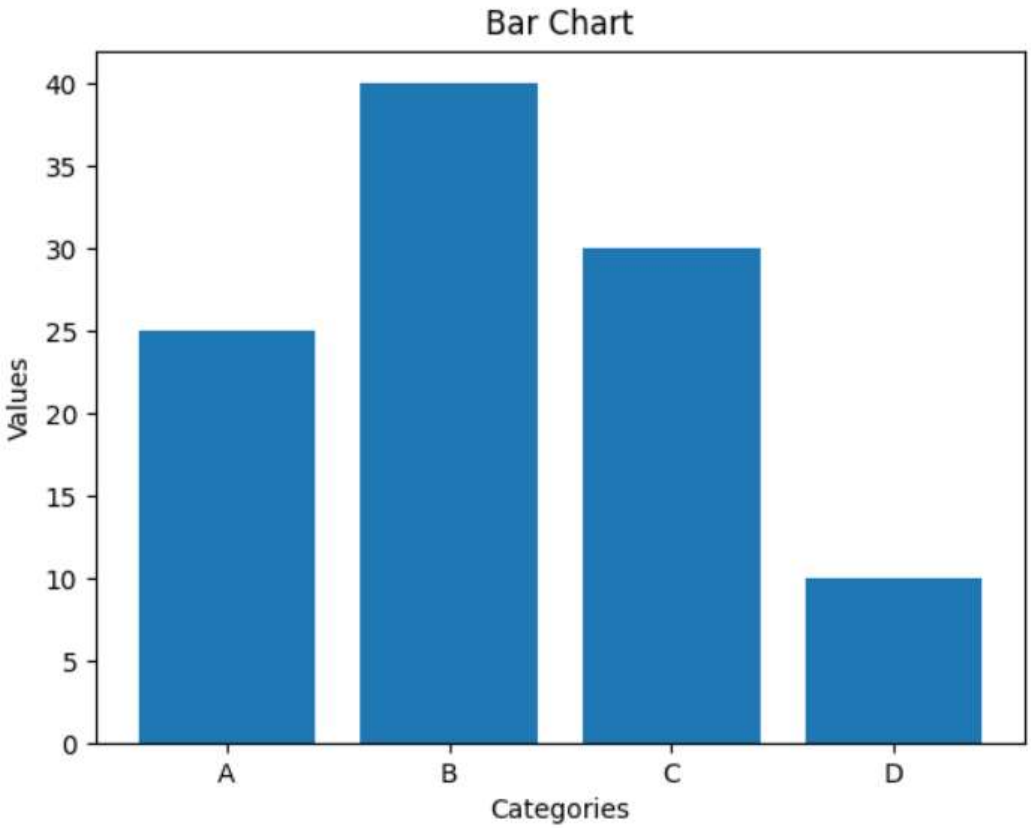
- **Importance of Data Visualization**
- Data visualization is essential for understanding and communicating information effectively. Here are some key reasons why it's important:
- **Simplifies Complex Data:** It turns large and complicated data into visual formats like charts and graphs, making the information easier to understand.
- **Reveals Patterns and Trends**: It helps identify trends, relationships and patterns that are not easily seen in raw data or tables.
- **Saves Time:** Visuals allow quicker interpretation of data, helping users spot key information at a glance instead of manually scanning through numbers.
- **Improves Communication:** It makes it easier to explain data insights to others, especially those who may not be familiar with the technical details.
- **Tells a Clear Story**: Data visuals guide the audience through the information step-by-step, making it easier to reach conclusions and make informed decisions.

- **Real-World Use Cases for Data Visualization**
- Data visualization is used across various industries to improve decision-making and drive results. Here are a few examples:
- **Business Analytics:** Used to monitor company performance, track KPIs and make data-driven decisions by visualizing trends, sales and customer metrics.
- **Healthcare:** Helps in analyzing patient records, tracking disease outbreaks and managing hospital operations through easy-to-read charts and dashboards.
- **Sports:** Used to visualize player statistics, team performance and match outcomes, helping coaches and analysts improve strategies and training plans.
- **Retail and E-commerce:** Enables tracking of sales, customer preferences and inventory levels, helping businesses adjust stock and marketing efforts effectively.

- **Challenges in Data Visualization**
- **Data Quality**: Accuracy of visualizations depends on the quality of the data. If the data is inaccurate or incomplete, the insights from the visualization will be misleading.
- **Over-Simplification**: Simplifying data too much can lead to important details being lost like using a pie chart that oversimplifies complex relationships between categories.
- **Choosing the Right Visualization**: Using the wrong type of visualization can distort the message. For example, a pie chart might not work well with many categories which leads to confusion.
- **Overload of Information**: Too much information in a visualization can overwhelm viewers. It's important to focus on key data points and avoid clutter

Different types of plots

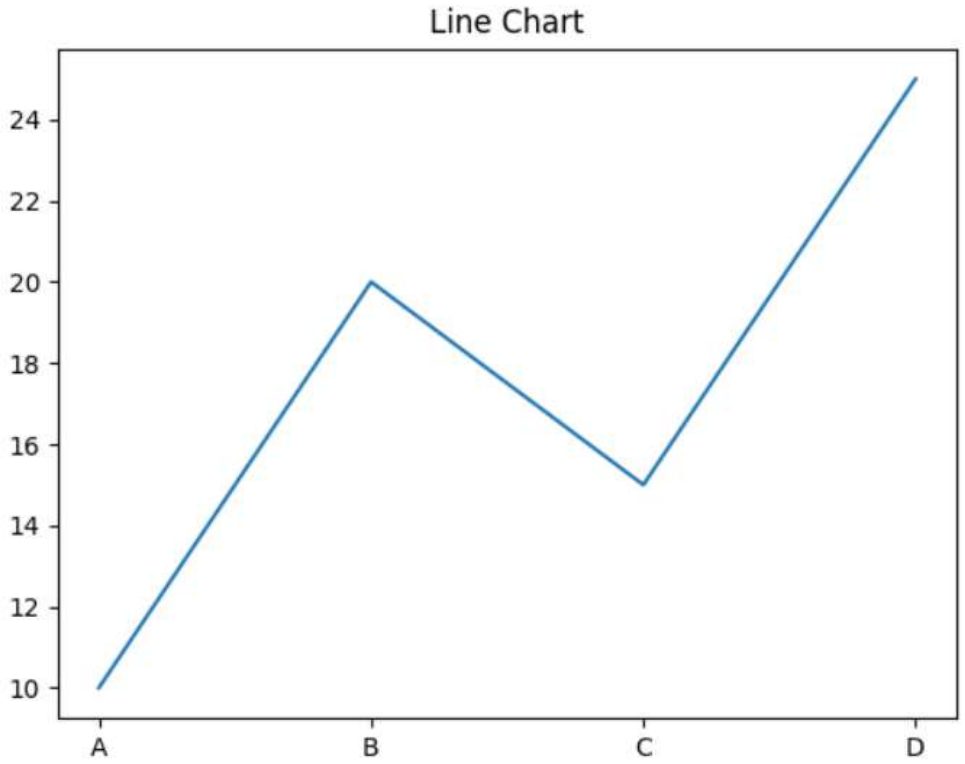
- **1. Bar Charts**
- Bar charts are used to compare values across different categories using rectangular bars. X-axis shows categories while Y-axis represents values. Common types include horizontal, stacked and grouped bar charts.
- Below is the Example of Bar Chart:



Representation of Bar Chart

- **When to Use:**
- To compare different categories
- To rank values from highest to lowest
- To show relationships between multiple variables

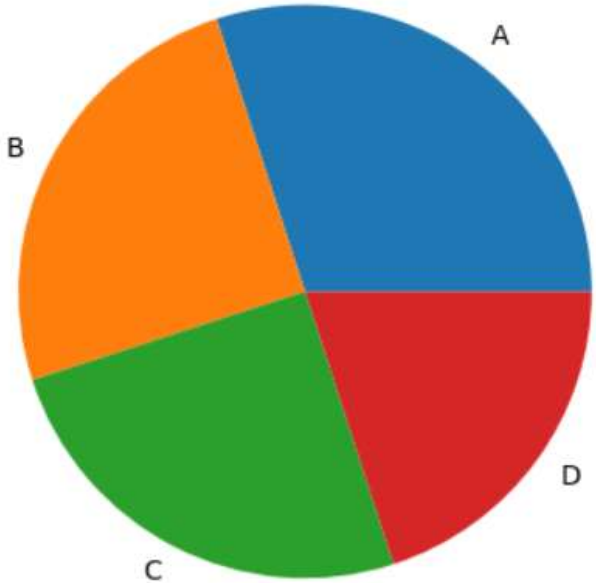
- **2. Line Charts**
- Line charts show how values change over time by connecting data points with lines. They help visualize trends like increases, decreases or stability.
- Below is the example of line chart:



Representation of line chart

- **When to Use:**
- To track changes over time
- To compare trends across multiple data series
- For time series analysis

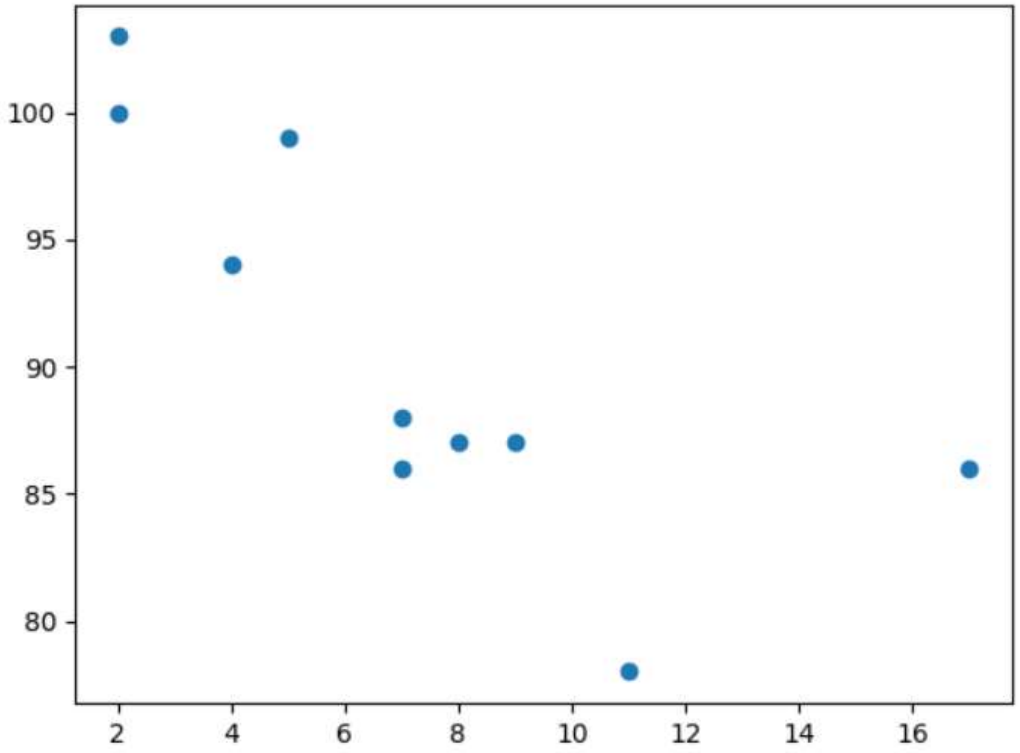
- **3. Pie Charts**
- Pie charts are round charts divided into slices, where each slice shows a part of the whole. The size of each slice represents its percentage.
- Below is the example of pie chart:
-



Representation of pie chart

- **When to Use:**
- To show how different parts contribute to a whole
- To highlight a dominant category

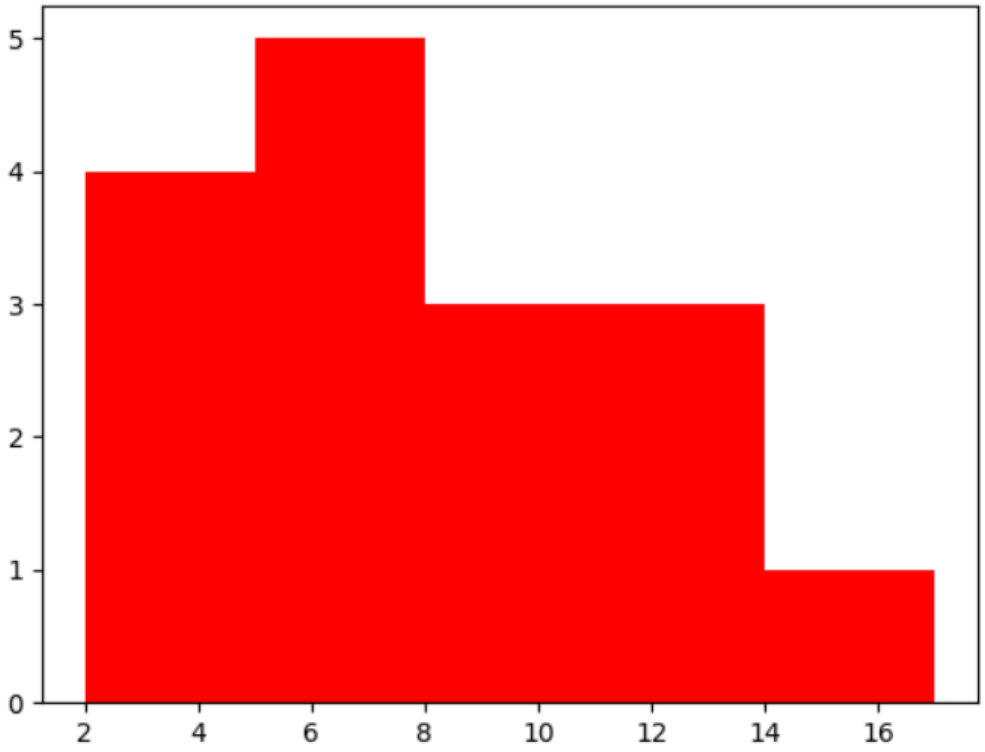
- **4. Scatter Chart (Plots)**
- Scatter charts use dots to show relationship between two numerical variables. X-axis shows the independent variable and Y-axis shows the dependent variable.
- Below is the example of scatter chart:



Representation of Scatter Chart

- **When to Use:**
- To observe relationships between two variables
- To detect patterns, clusters or outliers in data

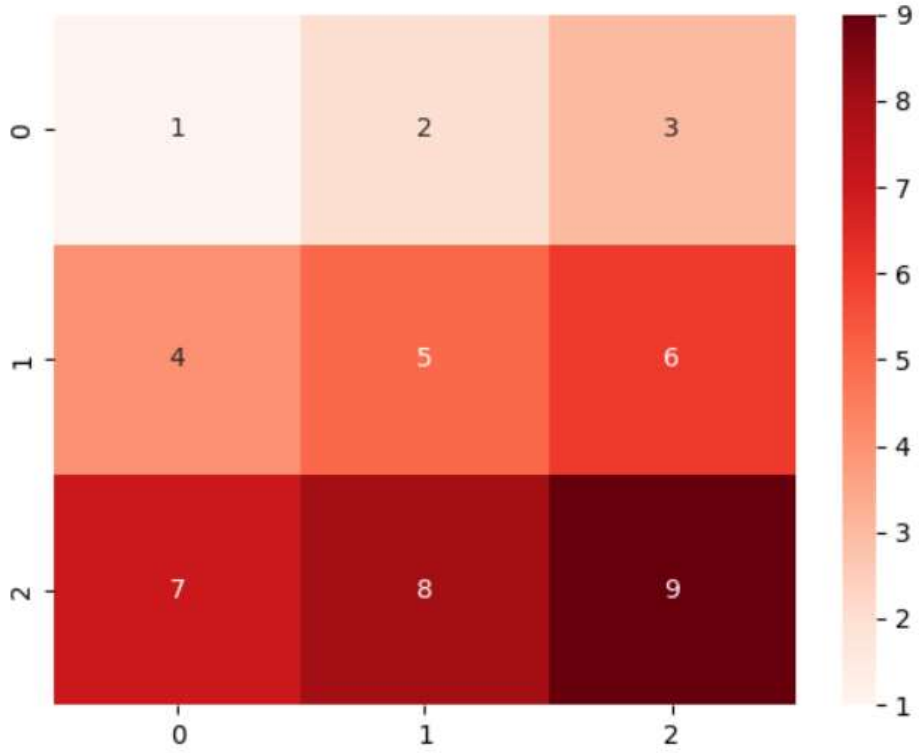
- **5. Histogram**
- A histogram displays the distribution of numerical data by grouping values into intervals (bins) and showing their frequency as bars. It helps reveal the shape, spread and patterns in the data.
- Below is the example of histogram:



Representation of Histogram

- **When to Use:**
- To visualize the distribution of numerical data
- To explore patterns, trends and outliers

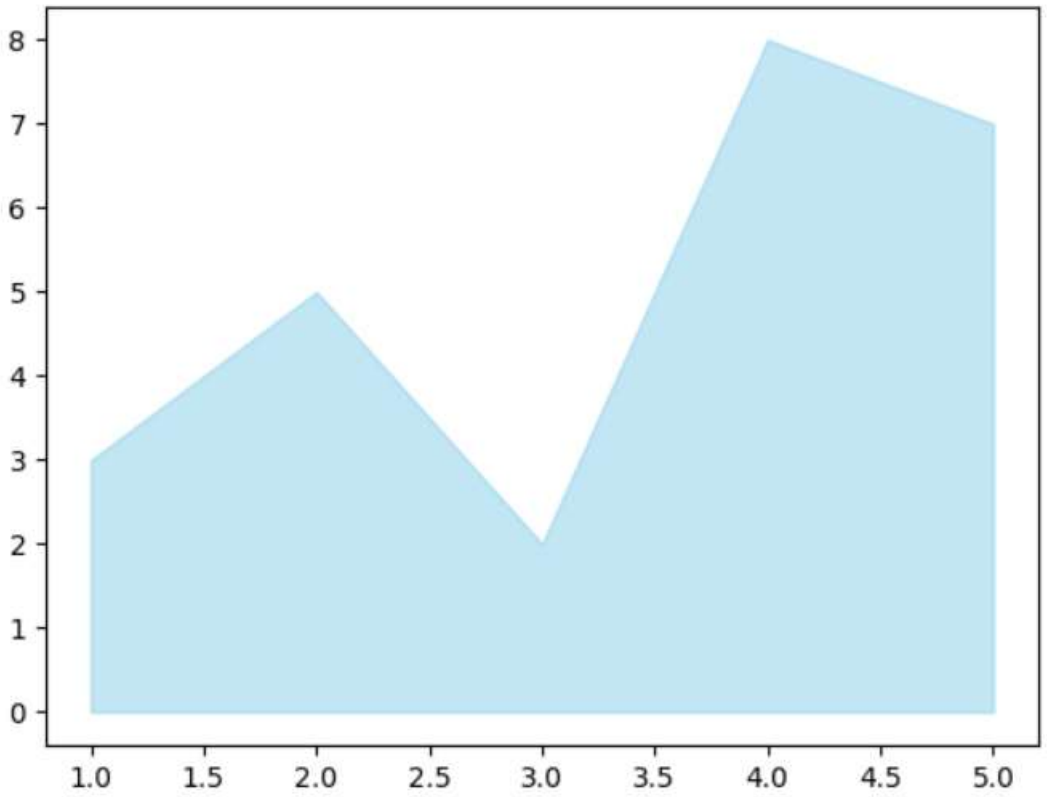
- **Advanced Charts for Data Visualization**
- Advanced charts are designed to handle more complex data. They help analyze multiple variables, uncover deeper insights and reveal patterns that might be missed with basic visuals.
- **1. Heatmap**
- A heatmap displays data in a matrix format using color to represent values. It's ideal for spotting patterns, correlations and variations in large datasets.
- Below is the example of heatmap:
-



Representation of Heatmap

- **When to Use:**
- To identify clusters or groupings in data
- To visualize correlations between variables
- For risk analysis in fields like finance or network security

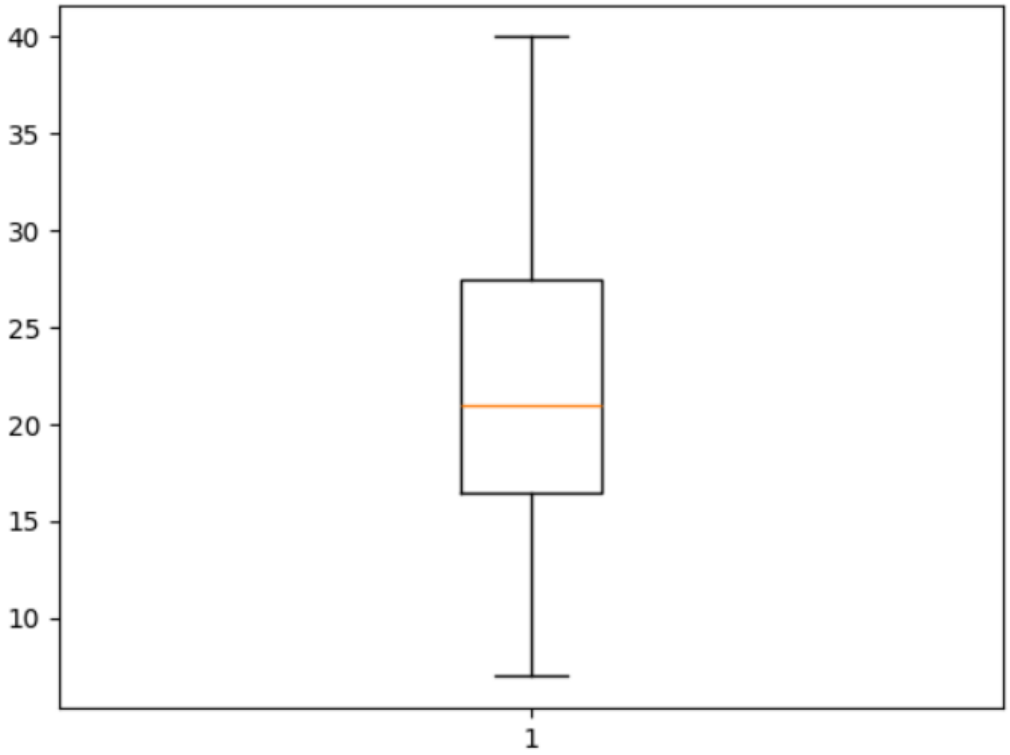
- **2. Area Chart**
- An area chart shows trends over time by filling the space beneath a line. It's ideal for visualizing time-series data and highlighting changes across periods.
- Below is the example of area chart:



Representation of Area chart

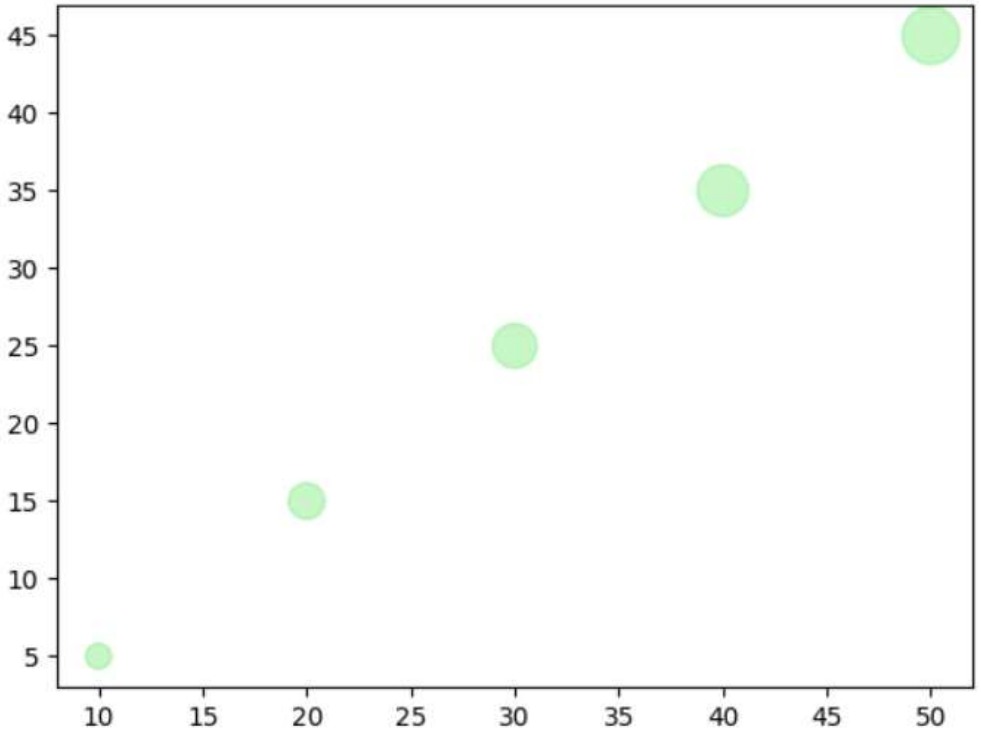
- **When to Use:**
- To track changes or trends over time
- To compare multiple data series
- To highlight patterns like seasonality or cycles

- **3. Box Plot (Box-and-Whisker Plot)**
- A box plot displays distribution of numerical data, showing median, quartiles and outliers. It's useful for understanding variability and detecting unusual values.
- Below is the example of box plot:



Representation of Box plot

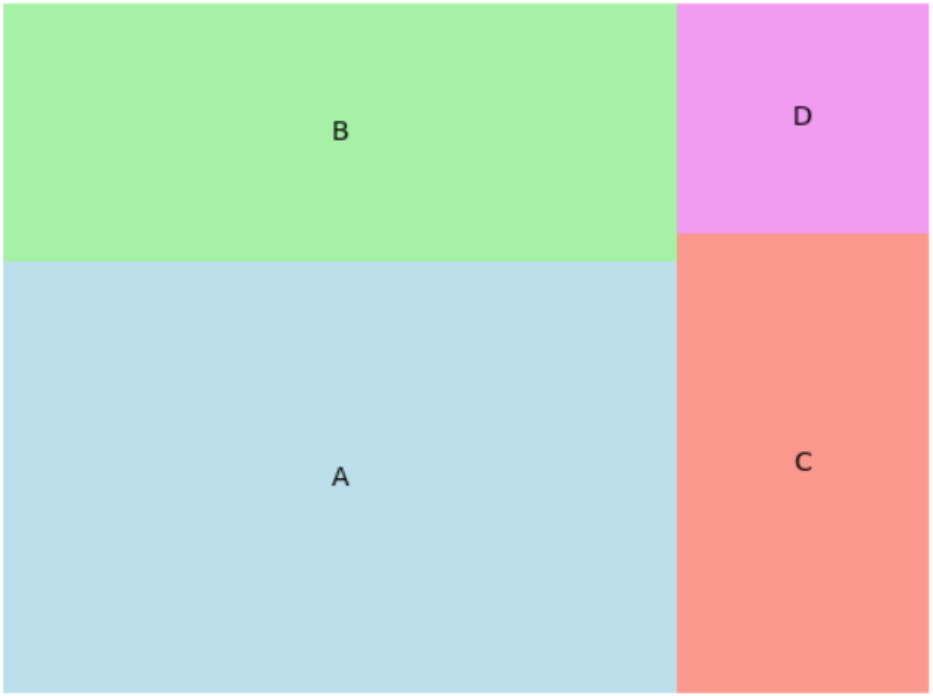
- **4. Bubble Chart**
- A bubble chart displays data points as circles, where size and color of each bubble represent additional variables. It's useful for visualizing three or more dimensions in a single chart.
- Below is the example of bubble chart:



Representation of Bubble chart

- **When to Use:**
- To compare multiple variables at once
- To represent data using size and color
- To visualize relationships and detect patterns

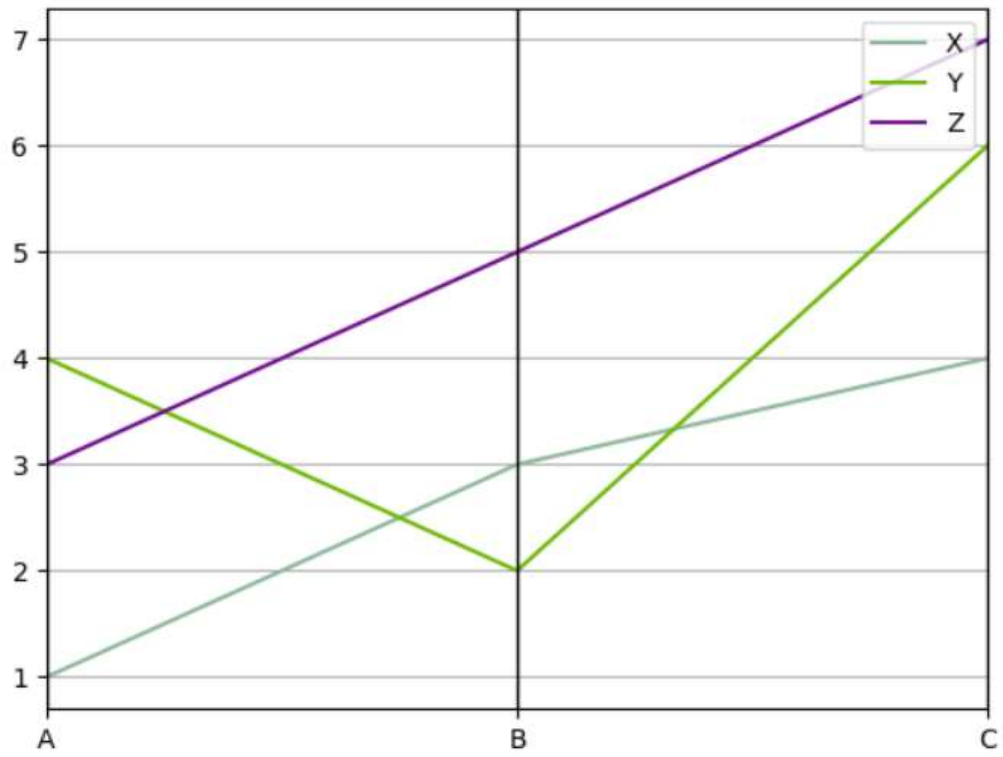
- **5. Tree Map**
- A tree map visualizes hierarchical data using nested rectangles, where the size of each represents a value. It's useful for showing structure and comparing proportions within a hierarchy.
- Below is the example of tree map:



Representation of Tree map

- **When to Use:**
- To display hierarchical data
- To compare proportions within levels
- To visualize large datasets compactly

- **6. Parallel Coordinates**
- Parallel coordinates display multivariate data using lines that connect values across multiple axes. Each line represents a data point across several variables.
- Below is the example of parallel coordinates:

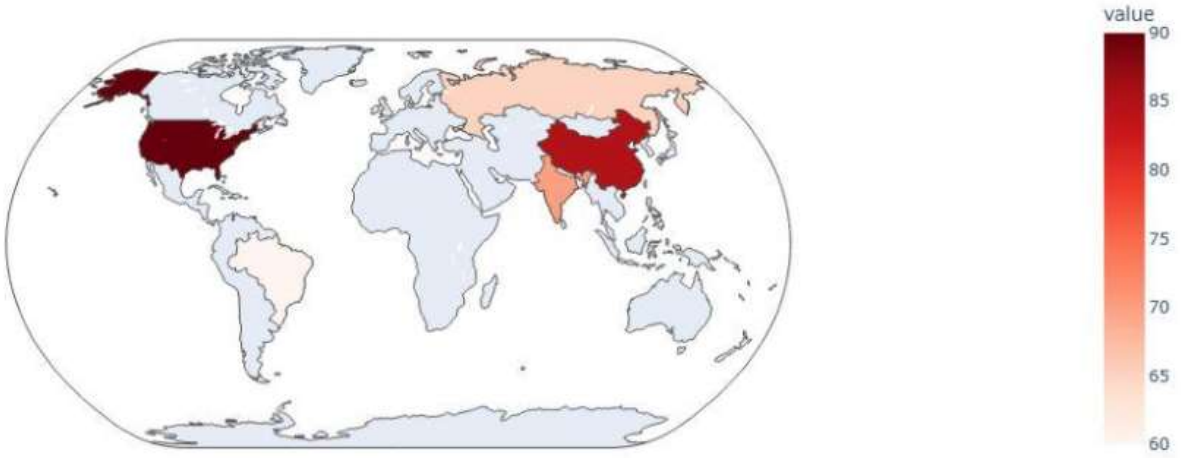


Representation of Parallel coordinates

- **When to Use:**
- To analyze multiple variables at once
- To visualize relationships or clusters
- To detect outliers in the data

- **7. Choropleth Map**
- A choropleth map uses color shading to represent data across geographic areas. It's useful for showing differences like population density, income levels or disease spread across regions.
- Below is the example of choropleth map:

Choropleth Map

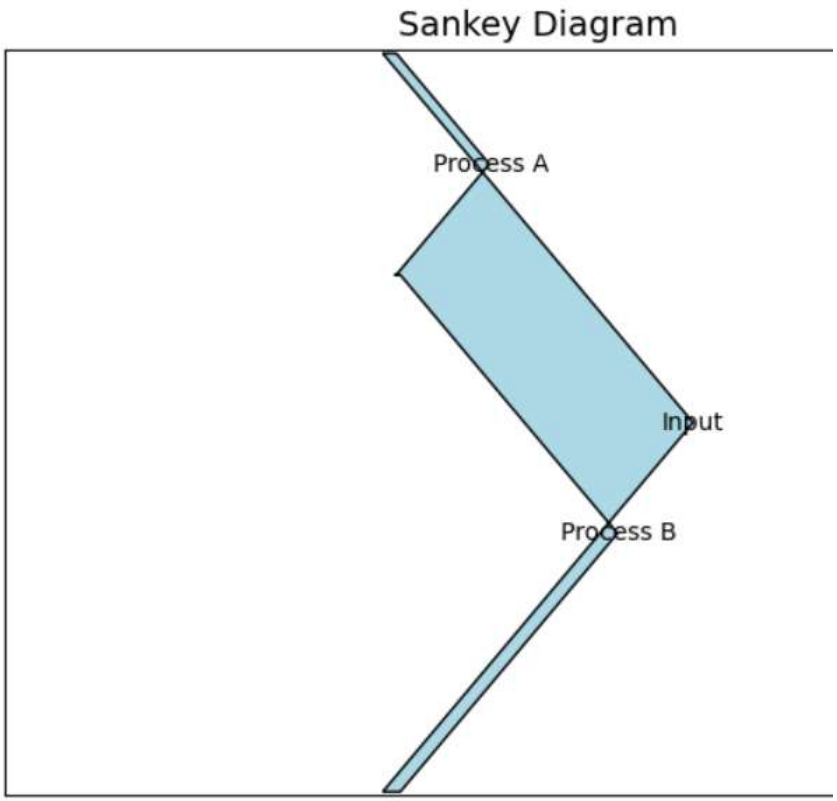


Representation of choropleth map

When to Use:

- To compare data across locations
- To spot geographic patterns and trends

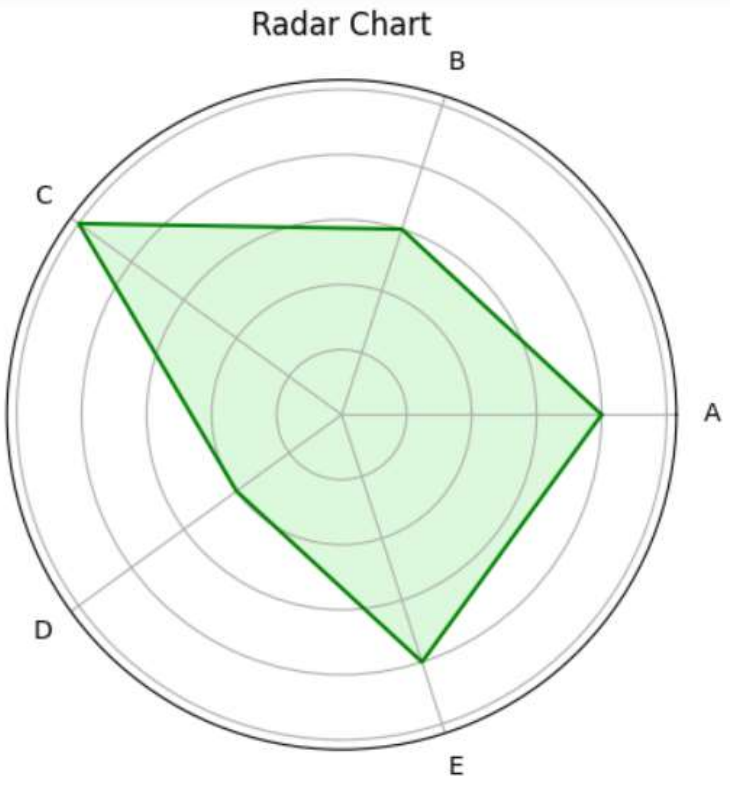
- **8. Sankey Diagram**
- A Sankey diagram shows the flow of data or resources between points (nodes) using arrows, where the width of each arrow represents the amount of flow. It's great for visualizing complex systems and spotting inefficiencies.
- Below is the example of Sankey diagram:



Representation of Sankey Diagram

- **When to Use:**
- Visualize Flows: Understand how resources move through a process.
- Find Bottlenecks: Identify points where flow slows or drops.
- Compare Scenarios: Track flow changes across time or conditions.
-

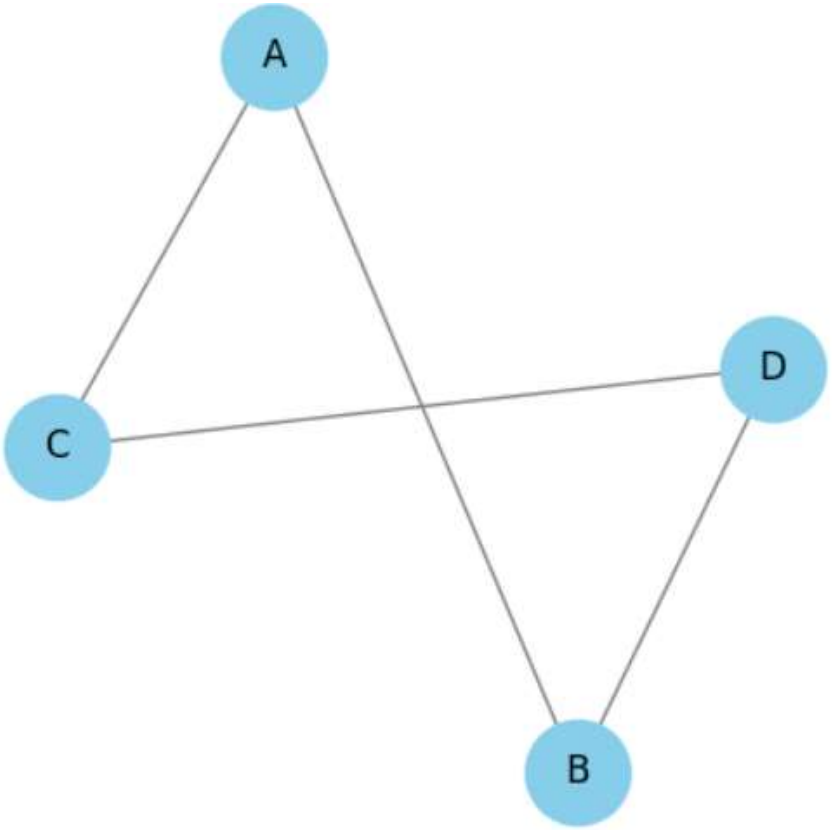
- **9. Radar Chart (Spider Chart)**
- A radar chart displays multivariate data using axes from a central point ideal for comparing variables across categories. Common in performance analysis, sports and decision-making.
- Below is the example of Radar Chart:



Representation of radar/spider chart

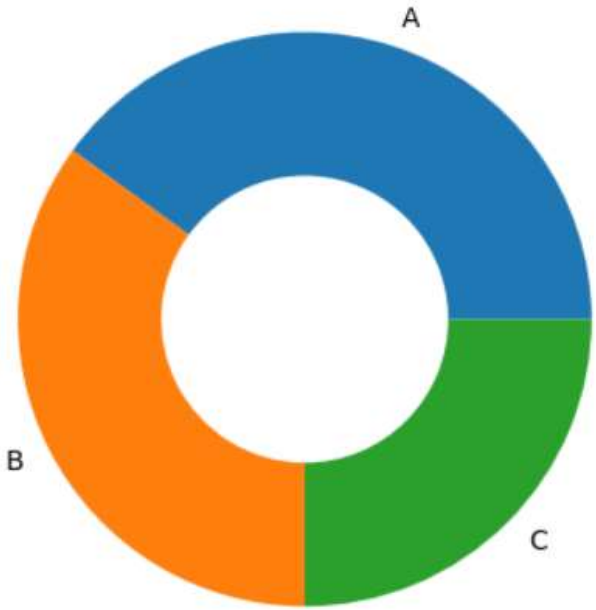
- **When to Use:**
- Multi-Criteria Comparison: Compare multiple variables at once.
- Strengths & Weaknesses: Visualize strong and weak areas.
- Pattern Recognition: Spot similarities or differences between categories.

- **10. Network Graph**
- A network graph shows relationships between entities as nodes and links (edges). It helps visualize complex networks like social media, transport routes or biological systems.
- Below is the example of a network graph:



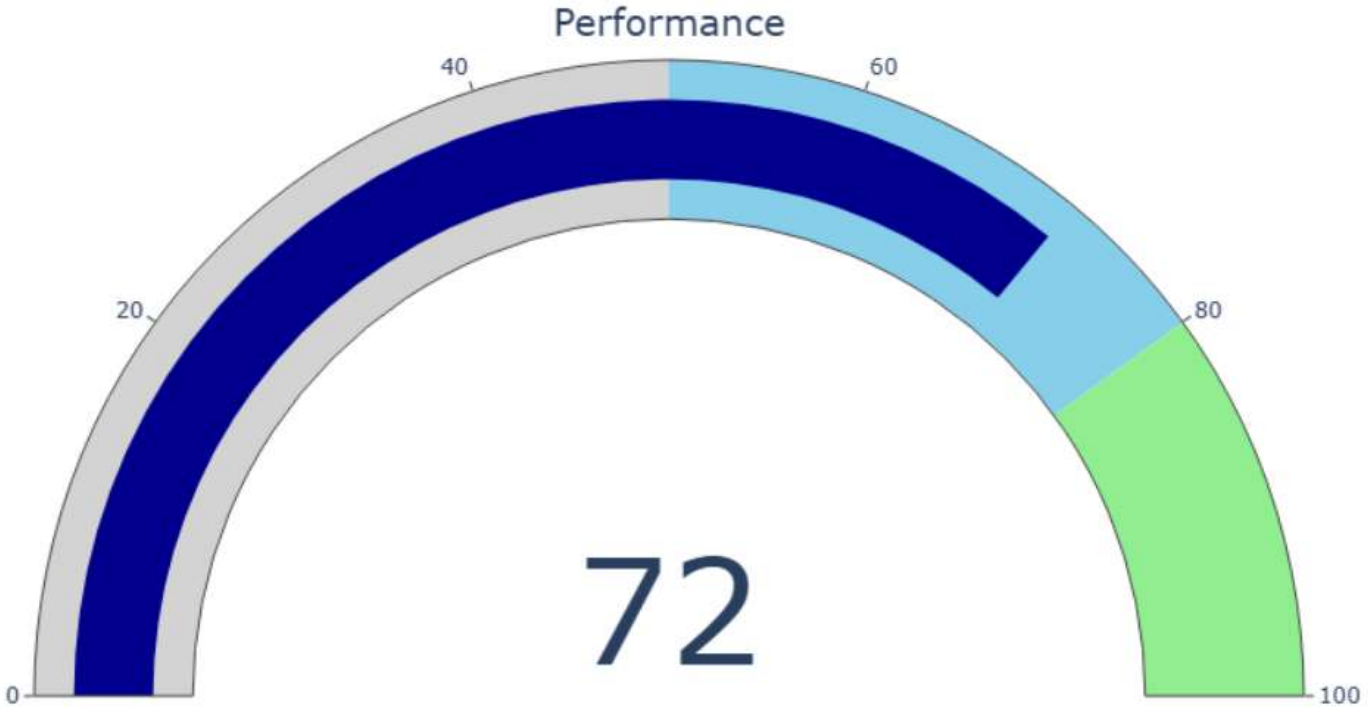
Representation of Network graph

- **When to Use:**
- Relationship Visualization: Helps show connections or interactions between entities.
- Community Detection: Useful to identify clusters or groups within a network.
- Path Analysis: Shows shortest or most efficient routes between nodes.
- **11. Donut or Doughnut chart**
- A donut chart is a circular chart like a pie chart but with a hole in the center. Each slice shows a category's contribution to the whole, making it visually cleaner and ideal for comparisons.
- Below is the example of a donut chart:

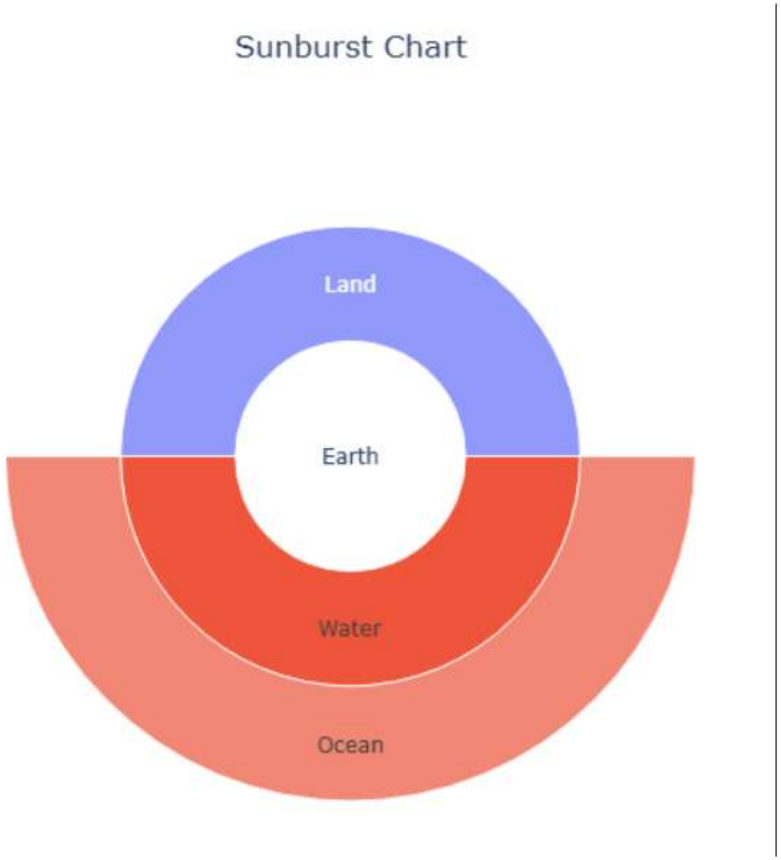


Representation of Donut Chart

- **When to Use:**
- Show part-to-whole relationships like sales or market share.
- Track progress toward a goal (e.g., 70% completed).
- Best when comparing a few categories.
- **12. Gauge Chart**
- A Gauge chart shows progress toward a goal using a dial-like arc, similar to a speedometer. It is useful for tracking a single value like a KPI or project status.
- Below is the example of Gauge chart:

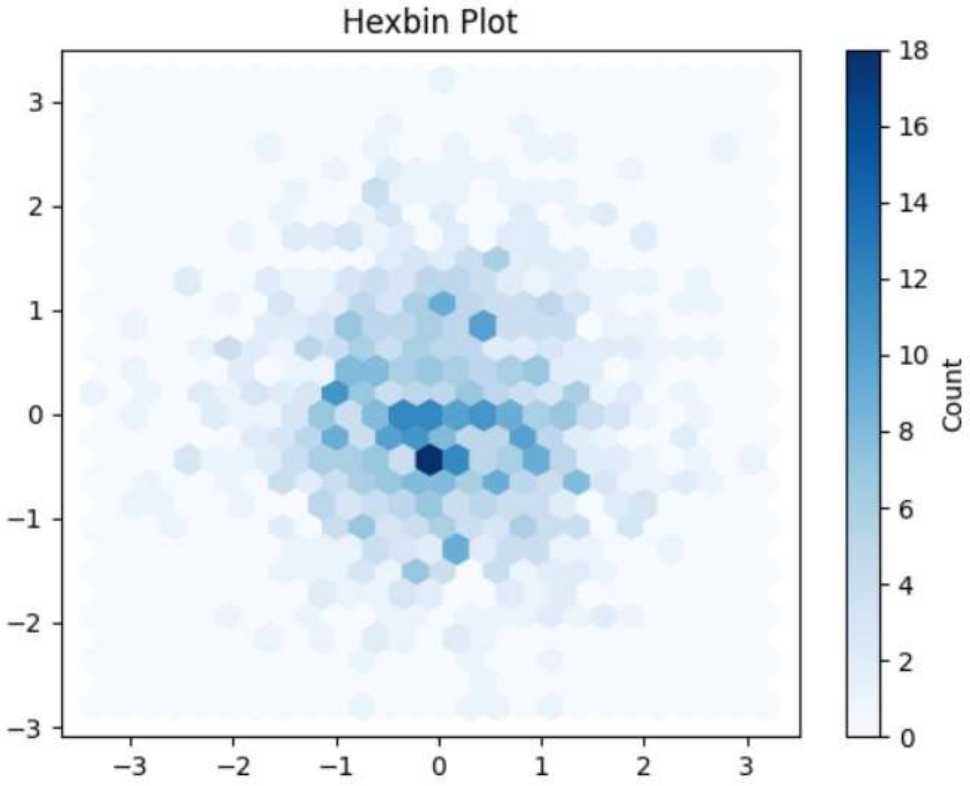

Performance
0
20
40
60
80
100
72

- **When to Use:**
- To monitor key metrics (e.g., sales, satisfaction) toward targets
- In KPI dashboards to track progress
- For project status tracking against deadlines
- **13. Sunburst Chart**
- A sunburst chart shows hierarchical data as nested rings. Each ring represents a level in the hierarchy making it useful for visualizing multi-level data structures like categories, subcategories or organizational hierarchies.
- Below is the example of sunburst chart:



Representation of Sunburst Chart

- **When to Use:**
- To visualize hierarchical structures like org charts or file systems.
- To explore proportions and relationships within nested datasets.
- To simplify complex data in a clean, interactive layout.
- **14. Hexbin Plot**
- A hexbin plot groups points into hexagonal bins and colors them by how many points fall in each bin. It's useful for large datasets to show density clearly without overplotting.
- Below is the example of a hexbin plot:



Representation of Hexbin Plot

- **When to Use:**
- To show data density in a 2D space.
- To spot clusters or patterns in large scatter data.
- To manage overlapping points in big datasets visually.
- **15. Violin Plot**
- A violin plot combines a box plot and a density plot to show distribution and summary statistics of data. It helps visualize the shape, spread and center of the data across categories.
- Below is the example of violin plot:

Representation of Violin plot

- **When to Use:**
- Compare distribution of continuous data across groups.
- Visualize data shape, spread, skewness or multimodal patterns.
- Highlight detailed distribution and outliers in a clean layout

Plotting fundamentals using Matplotlib

- Matplotlib is an open-source **visualization library** for the Python programming language, widely used for creating **static**, **animated** and **interactive plots.** It provides an **object-oriented API** for embedding plots into applications using general-purpose GUI toolkits like Tkinter, Qt, GTK and wxPython. It offers a variety of plotting functionalities, including line plots, bar charts, histograms, scatter plots and 3D visualizations. Created by John D. Hunter in 2003, Matplotlib has become a fundamental tool for data visualization in Python, extensively used by data scientists, researchers and engineers worldwide.

Important Facts to know:

- ***Matplotlib Pyplot:*** *The pyplot module is a collection of functions that make Matplotlib work like MATLAB, providing a simple interface for creating plots.*
- ***Figure and Axes:*** *In Matplotlib, figures represent the overall container, while axes refer to the individual plots within a figure.*
- ***Integration with Pandas:*** *Matplotlib works seamlessly with Pandas DataFrames, enabling efficient data visualization.*

- **What is Matplotlib in Python used for?**
- With Matplotlib, we can perform a wide range of visualization tasks, including:
- Creating basic plots such as line, bar and scatter plots.
- Customizing plots with labels, titles, legends and color schemes.
- Adjusting figure size, layout and aspect ratios.
- Saving plots in various formats like PNG, PDF and SVG.
- Combining multiple plots into subplots for better data representation.
- Creating interactive plots using the widget module.

- **Toolkits in Matplotlib**
- Several toolkits extend Matplotlib's functionality, some of which are external downloads, while others are included with Matplotlib but have external dependencies. Here are some of the most notable toolkits:
- **Seaborn**: A high-level statistical data visualization library built on top of Matplotlib, extremely popular for creating attractive and informative statistical graphics with minimal code.
- **Mplot3d**: Integrated into Matplotlib itself, this toolkit is the go-to choice for creating 3-D plots with ease and flexibility.
- **GeoPandas**: A library that leverages Matplotlib for geospatial plotting, simplifying the handling of geospatial data without needing a spatial database.
- **Cartopy**: A modern mapping library offering an object-oriented approach to map projections and geospatial data, largely replacing Basemap in new projects.
- **Tikzplotlib**: A niche toolkit that converts Matplotlib figures into LaTeX-friendly TikZ/PGFPlots code, ideal for producing high-quality, publication-ready plots.

Plotting fundamentals using Seaborn

Seaborn is a Python library built on top of Matplotlib that focuses on statistical data visualization. It provides high-level functions, built-in themes, and automatic handling of datasets, allowing users to create informative and visually appealing plots with minimal code. Seaborn is widely used for exploring trends, relationships and distributions in data.

Popular Plots in Seaborn

Scatter Plot

Used to analyze the relationship between two numerical variables.

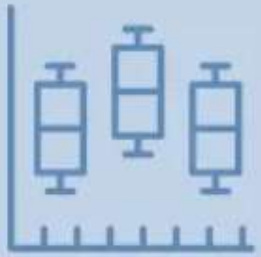
Line Plot

Great for tracking trends and changes over time or continuous data.

Popular Plots in Seaborn

Heatmap

Displays data values using color gradients, ideal for correlations.



Boxplot

Summarizes distribution using quartiles and highlights outliers.

Popular Plots in Seaborn

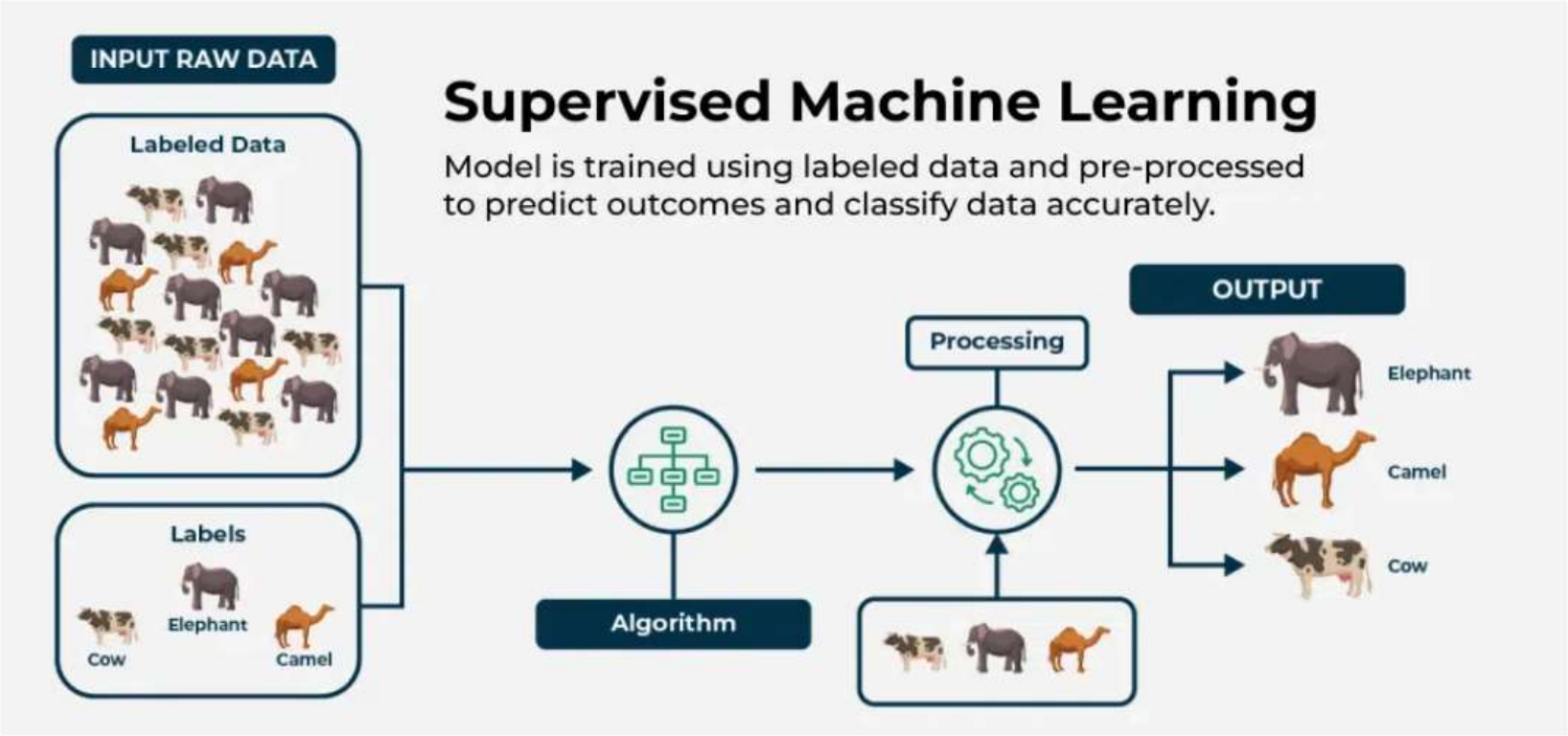
Bar Plot

Shows comparisons across categories using aggregated values.

Histogram / KDE Plot

Reveals the distribution, spread, and shape of your data.

Supervised Learning Techniques:



Supervised Machine Learning

Regression in machine learning

- Regression in machine learning is a supervised learning technique used to predict continuous numerical values by learning relationships between input variables (features) and an output variable (target). It helps understand how changes in one or more factors influence a measurable outcome and is widely used in forecasting, risk analysis, decision-making and trend estimation.
- Works with real-valued output variables
- Helps to identify strengths and the type of relationships
- Supports both simple and complex predictive models.
- Used for tasks like price prediction, trend forecasting and risk scoring.

Types of Regression

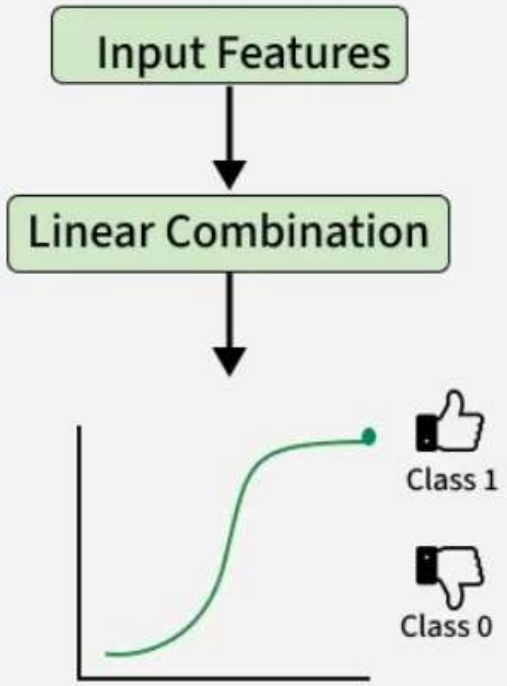
- Regression can be classified into different types based on the number of predictor variables and the nature of the relationship between variables:
- **1. Simple Linear Regression**
- Simple Linear Regression models the relationship between one independent variable and a continuous dependent variable by fitting a straight line that minimizes the sum of squared errors. It assumes a constant rate of change, meaning the output varies proportionally with the input.
- **Application:** Estimating house price from only its size
- **Advantage:** Highly interpretable due to its simple mathematical structure
- **Disadvantage:** Cannot capture curved or complex data patterns

2. Multiple Linear Regression

- Multiple Linear Regression extends simple linear regression by incorporating multiple independent variables to predict a continuous outcome. Each predictor is assigned a coefficient that reflects its individual impact while holding other variables constant.
- **Application:** Predicting house prices using multiple factors like size, location, age and number of rooms
- **Advantage:** Captures the combined influence of many factors simultaneously
- **Disadvantage:** Performance drops in the presence of multicollinearity (features highly correlated with each other)

What is Logistic Regression?

- Predicts the probability of a binary outcome (Yes/No, 0/1)
- Uses the sigmoid function to map inputs to probabilities (0 to 1)
- Ideal for classification tasks



- **3. Polynomial Regression**
- Polynomial Regression models non-linear relationships by transforming input features into higher-degree polynomial terms (e.g x^2, x^3). Although it fits non-linear curves it remains a linear model in terms of parameters.
- **Application:** Modelling curved growth trends like population increase or temperature variation
- **Advantage:** Effectively captures non-linear relationships without switching to non-linear algorithms
- **Disadvantage:** Higher-degree polynomials may lead to overfitting and unstable predictions

- **4. Ridge and Lasso Regression**
- Ridge and Lasso are regularized linear regression techniques that add penalty terms to limit large coefficients and reduce overfitting. Ridge (L2) shrinks coefficients smoothly, while Lasso (L1) can reduce some coefficients to zero, enabling feature selection.
- **Application:** Used in high-dimensional datasets like marketing attribution or gene expression data
- **Advantage:** Controls overfitting and improves generalization, especially with many predictors
- **Disadvantage:** Penalty terms make model interpretation less straightforward

Regularization:

- Regularization is a technique used in machine learning to prevent overfitting, which otherwise causes models to perform poorly on unseen data. By adding a penalty for complexity, regularization encourages simpler and more generalizable models.
- **Prevents overfitting:** Adds constraints to the model to reduce the risk of memorizing noise in the training data.
- **Improves generalization:** Encourages simpler models that perform better on new, unseen data.

- **Types of Regularization**
- There are mainly 3 types of regularization techniques, each applying penalties in different ways to control model complexity and improve generalization.
- **1. Lasso Regression**
- A regression model which uses the L1 Regularization technique is called LASSO (Least Absolute Shrinkage and Selection Operator) regression. It adds the absolute value of magnitude of the coefficient as a penalty term to the loss function(L). This penalty can shrink some coefficients to zero which helps in selecting only the important features and ignoring the less important ones.
- Where
- m: Number of Features
- n: Number of Examples
- y_i :Actual Target Value
- $\hat{y}_i$:Predicted Target Value

- **2. Ridge Regression**
- A regression model that uses the L2 regularization technique is called Ridge regression. It adds the squared magnitude of the coefficient as a penalty term to the loss function(L). It handles multicollinearity by shrinking the coefficients of correlated features instead of eliminating them.
- Where,
- n: Number of examples or data points
- m: Number of features i.e predictor variables
- y_i :Actual target value for the ith example
- $\hat{y}_i$:Predicted target value for the ith example
- w_i :Coefficients of the features
- λ: Regularization parameter that controls the strength of regularization

Classification: Binary Classification

- Binary classification is a **supervised machine learning** task where the goal is to categorize input data into **one of two possible classes**.
- **Examples**
- Spam detection: *Spam* or *Not Spam*
- Medical diagnosis: *Disease* or *No Disease*
- Sentiment analysis: *Positive* or *Negative*
-

and Multi-Class Classification,

- **Step-by-Step Implementation**
- Let's see the step-by-step implementation of Multiclass Classification along with various classifiers,
- **Step 1: Import Libraries**
- We will import the required libraries,
- **sklearn.datasets**: Provides standard datasets (like iris) useful for testing and practicing ML methods.
- **sklearn.model_selection.train_test_split**: This function splits arrays or matrices into random train and test subsets, enabling fair evaluation of models.
- **sklearn.metrics.accuracy_score, confusion_matrix**: Tools for evaluating the correctness of model predictions; accuracy measures percent correct, confusion matrix details classification mistakes.
- **matplotlib.pyplot**: A plotting library for creating static, interactive and animated visualizations in Python.
- **seaborn**: A high-level data visualization library built on matplotlib; it helps produce visually appealing statistical graphics (like heatmaps).

- **Step 2: Load and Explore the Dataset**
- The Iris dataset is a famous collection of 150 flower samples, representing three Iris species, setosa, versicolor and virginica. Each sample has four numeric features: sepal length, sepal width, petal length and petal width.
- **iris.data** is a 2D NumPy array of shape (150, 4), where each row represents a single flower's measured features.
- **iris.target** is a 1D array of length 150, where each entry is an integer (0, 1 or 2) that denotes the species label for the corresponding row in iris.data.

- **Step 3: Split the Data**
- We will split the data for training and testing,
- train_test_split separates the feature (X) and label (y) arrays into training and testing sets. Here, 70% of the data is used to train the models (X_train, y_train) and 30% is used to evaluate them (X_test, y_test).
- Setting random_state ensures we always get the same split, allowing reproducibility

- **Step 4: Model Training and Visualization**
- **1. Decision Tree Classifier:** Decision Tree Classifier is a model that predicts class labels by learning simple decision rules arranged in a tree structure, where each node makes a decision based on a feature until a class label is assigned at the leaf.
- Instantiates the classifier object, setting a limit of 2 for tree depth.
- .fit(X_train, y_train) trains the model using the training data.
- .predict(X_test) generates predicted labels for the test data.
- accuracy_score(y_test, dtree_preds) computes how many test samples were correctly classified.
- confusion_matrix(y_test, dtree_preds) gives a table indicating, for each actual class, how many times the model predicted each possible class.
- The seaborn heatmap visualizes which classes the model predicts well or struggles with.

Support Vector Machine,

- **2. Support Vector Machine(SVM) Classifier:** Support Vector Machine Classifier is a model that separates data into classes by finding the optimal hyperplane that maximizes the margin between different class groups in the feature space.
- Creates a linear SVC (Support Vector Classifier) object.
- Fits the model with training data.
- Predicts test set labels.
- Calculates accuracy and confusion matrix.
- Visualizes the confusion matrix, showing per-class predictions.

K-Nearest Neighbours,

- **3. K-Nearest Neighbors(KNN) Classifiers:** k-Nearest Neighbors Classifier is a model that classifies a data point by looking at the majority class among its k-nearest neighbors, based on distance in feature space.
- Sets up a KNN classifier to consider 7 neighbors.
- Models the training data (essentially, stores it).
- Predicts the labels by 'voting' among the nearest neighbors.
- Derives accuracy and confusion statistics.
- Plots the confusion matrix to visualize strengths and errors in class assignment.

Naive Bayes classifier

- **4. Naive Bayes Classifier:** Naive Bayes Classifier is a probabilistic model based on Bayes' theorem, which assumes that features are independent given the class and predicts the most probable class for new data.
- Instantiates a Gaussian Naive Bayes classifier.
- Fits the model to the training data, calculating class-conditional mean and variance for each feature.
- Predicts class labels for test samples based on probability.
- Computes overall accuracy and structured confusion data.
- Plots the confusion matrix, providing an at-a-glance assessment of classification quality for each true class.

Decision Trees

Decision Tree is very popular supervised machine learning algorithm used for regression as well as classification problems. In decision tree, a flow-chart like structure is build where each internal nodes denotes the features, rules are denoted using the branches and the leaves denotes the final result of the algorithm.

Random Forest

Random Forest is very powerful supervised machine learning algorithm, used for classification and regression task. Random Forest uses e**nsemble learning** (combining multiple models/classifiers to solve a complex problem and to improve the overall accuracy score of the model). In Random Forest multiple decision tree are built by considering the different subset of the given data and the average of all those to increase the overall accuracy of the model. As the number of decision tree in random forest increases the accuracy increases and overfitting also reduces.